

Graphe licence informatique

Document descriptif du TP/Projet Théorie des Graphes

Réalisé par :

Khadidja Ahmat Hassan

Djibrine Moustapha Mahamat

Enseignant :

Philippe Gaborit

Année Académique : 2024 - 2025

Exercice 1 : Composantes fortement connexes

L'objectif est de déterminer les composantes fortement connexes d'un graphe orienté. Une composante fortement connexe est un groupe de sommets où chaque sommet peut atteindre tous les autres, et être atteint en retour.

❖ Rappel du principe de l'algorithme utilisé

Nous avons utilisé une méthode par calcul matriciel. L'idée est de :

- Représenter le graphe avec une matrice d'adjacence.
- Calculer la fermeture transitive de cette matrice, c'est-à-dire déterminer tous les chemins possibles (directs ou indirects) entre les sommets.
- Identifier les sommets fortement liés, c'est-à-dire ceux qui peuvent s'atteindre mutuellement : ils forment une même composante.

Dans notre cas, la fermeture transitive est obtenue en multipliant la matrice avec elle-même jusqu'à ce qu'elle ne change plus. On ajoute aussi la matrice identité pour tenir compte des boucles (sommets liés à eux-mêmes). Ensuite, on cherche les sommets i et j tels que :

mat[i][j] == 1 ET mat[j][i] == 1

Cela signifie que i peut atteindre j **et** j peut atteindre i ⇒ ils sont dans la même composante.

❖ Explication de la structure de données utilisée :

La structure de données utilisée est une matrice d'adjacence. Nous avons utilisé une matrice carrée d'entiers pour représenter le graphe.

- Chaque ligne représente un sommet de départ.
- Chaque colonne représente un sommet d'arrivée.
- La valeur 1 indique l'existence d'une arête entre deux sommets.

Par exemple, **mat[0][2] = 1** signifie que le sommet A pointe vers le sommet C. Les noms des sommets (A, B, C, ...) sont représentés par des indices de 0 à 4 ou 5.

❖ Cas d'usage :

- Cas d'usage 1 : Un graphe vu en TD Graphe d'exemple :

```
----------------------------------Matrice Initiale-----------------------------------
0 1 0 1 0
0 1 1 0 0
0 0 0 0 1
0 0 1 0 1
0 0 0 1 1
---------------------------------Fermeture Transitive--------------------------------
1 1 1 1 1
0 1 1 1 1
0 0 1 1 1
0 0 1 1 1
0 0 1 1 1
---------------------------------Composantes Fortement Connexes---------------------
Les composantes fortement connexes sont :
[ A ]
[ B ]
[ C D E ]
PS C:\Users\ahmat\OneDrive\Documents\Tp_Théorie_des_graphes>
```

- Cas d'usage 2 : Un graphe de choix personnel graphe d'exemple :

```
----------------------------------Matrice Initiale-----------------------------------
0 1 0 0 0 0
0 0 1 0 0 0
1 0 0 0 0 0
0 0 0 0 1 0
0 0 0 1 0 0
0 0 0 0 0 0
---------------------------------Fermeture Transitive--------------------------------
1 1 1 0 0 0
1 1 1 0 0 0
1 1 1 0 0 0
0 0 0 1 1 0
0 0 0 1 1 0
0 0 0 0 0 1
---------------------------------Composantes Fortement Connexes---------------------
Les composantes fortement connexes sont :
[ A B C ]
[ D E ]
[ F ]
PS C:\Users\ahmat\OneDrive\Documents\Tp_Théorie_des_graphes>
```

Exercice 2 : Plus court chemin (**Algorithme de Dijkstra**)

❖ Rappel du principe de l'algorithme utilisé

L'algorithme de Dijkstra permet de trouver le plus court chemin entre un sommet de départ et tous les autres sommets dans un **graphe pondéré** (avec des poids) **sans poids négatif**. On peut dire de manière simple L'algorithme de Dijkstra trouve le plus court chemin entre deux sommets dans un graphe en utilisant une approche de recherche de plus petit coût.

Le principe est :

- On initialise toutes les distances à "infini" sauf celle du point de départ qui est 0.
- On sélectionne à chaque fois le sommet non encore visité avec la plus petite distance.
- On met à jour les distances de ses voisins si un chemin plus court est trouvé.

- On répète jusqu'à ce que tous les sommets aient été explorés.

❖ **Explication de la structure de données utilisée :**

Dans notre programme Java :

- Le graphe est représenté par une matrice d'adjacence int[][] graphe, où graphe[i][j] donne le poids entre les sommets i et j.
- Un tableau **distances[]** est utilisé pour stocker la distance minimale entre le sommet de départ et tous les autres.
- Un tableau **predecesseurs[]** permet de reconstituer le chemin à la fin.
- Un ensemble **nonVisites** garde les sommets qu'il reste à traiter.
- Les noms des sommets sont stockés dans un tableau **char[]** sommets = {'A', 'B', ..., 'G'} pour les affichages.

❖ **Cas d'usage :**

- Cas d'usage 1 : Graphe vu en TD

```
0 4 0 0 0 3 7
4 0 2 0 0 0 0
0 2 0 2 0 1 0
0 0 2 0 1 0 0
0 0 0 1 0 0 3
3 0 1 0 0 0 5
7 0 0 0 3 5 0
Plus court chemin de A à E : A --> F --> C --> D --> E
Distance totale : 7
PS C:\Users\ahmat\OneDrive\Documents\Tp_Théorie_des_graphes>
```

- Cas d'usage 2 : Un graphe de choix personnel Graphe d'exemple :

```
0 4 2 0 0 0
4 0 5 10 0 0
2 5 0 0 3 0
0 10 0 0 4 11
0 0 3 4 0 2
0 0 0 11 2 0
Plus court chemin de A à F : A --> C --> E --> F
Distance totale : 7
PS C:\Users\ahmat\OneDrive\Documents\Tp_Théorie_des_graphes>
```

Exercice 3 : Coloration de graphe **(algorithme de Welsh-Powell)**

❖ **Rappel du principe de l'algorithme**

L’algorithme de Welsh-Powell est une méthode de coloration de graphe. Son but est d’attribuer une couleur à chaque sommet de manière que deux sommets reliés entre eux (adjacents) n’aient jamais la même couleur.

Voici le principe :

- On commence par trier les sommets du graphe selon leur degré décroissant (c’est-à-dire, du sommet qui a le plus de voisins à celui qui en a le moins).
- On attribue la première couleur au sommet ayant le plus grand degré.
- On parcourt les autres sommets dans l’ordre décroissant et on leur attribue la même couleur s’ils ne sont pas adjacents aux sommets déjà colorés.
- On passe ensuite à une nouvelle couleur pour les sommets restants et on répète l’opération jusqu’à ce que tous les sommets soient colorés.

❖ **Explication de la structure de données utilisée :**

Le graphe est représenté sous forme de **matrice d’adjacence** int[][], où :

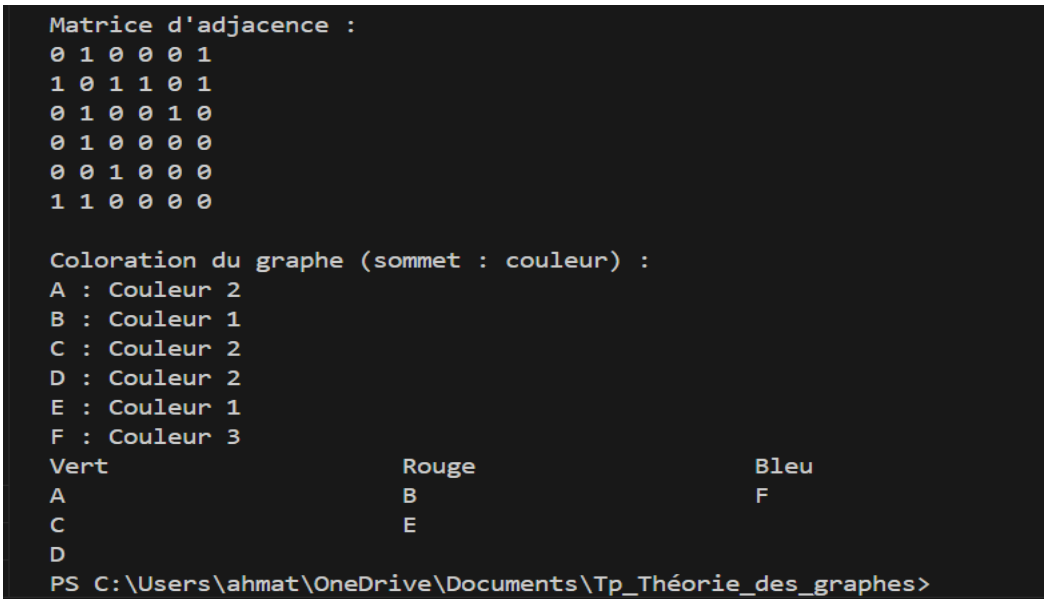
- graphe[i][j] = 1 signifie que les sommets i et j sont adjacents.
- graphe[i][j] = 0 signifie qu’ils ne sont pas reliés.

Les couleurs sont représentées par un tableau int[] couleurs, où chaque case donne la couleur attribuée à un sommet (par exemple : A = 1, B = 2, etc.).

Pour améliorer la lisibilité, on a aussi associé un nom de couleur (ex. : Rouge, Cyan, Jaune...) à chaque groupe de sommets colorés.

❖ **Cas d’usage :**

- Cas d’usage 1 : Graphe vu en TD



- Cas d'usage 2 : Un graphe de choix personnel graphe d’exemple :

```
Matrice d'adjacence :
0 1 1 0 0 0
1 0 1 1 0 0
1 1 0 0 0 0
0 1 0 0 1 1
0 0 0 1 0 1
0 0 0 1 1 0

Coloration du graphe (sommet : couleur) :
A : Couleur 2
B : Couleur 1
C : Couleur 3
D : Couleur 2
E : Couleur 1
F : Couleur 3
Vert                    Rouge                   Bleu
A                       B                       C
D                       E                       F
PS C:\Users\ahmat\OneDrive\Documents\Tp_Théorie_des_graphes>
```